LangChain 生态五大组件

定位差异与选型指南

LangGraph · LangChain · Deep Agents · Managed Deep Agents · LangSmith Fleet

技术对比 · 2026.10

控制粒度从高到低：五层架构一览

越往下自由度越高、代码量越大；越往上越开箱即用、代码越少



LangGraph · 底层编排引擎

定位

面向带状态、长时间运行的代理的底层运行时，支持持久执行、流式输出、状态记忆、人工介入。

核心能力

- 自定义任意复杂工作流、循环、分支
- 断点续跑、持久化状态
- 人工审批节点、中断恢复
- LangChain / Deep Agents 底层都基于它

适合场景

需要高度自定义流程、复杂分支逻辑、断点续跑、人工干预审批的生产级 Agent。

代价

代码量最大，需要理解图状态编排模型，开发和维护成本最高。

LangChain · 高层组件框架

定位

大模型应用的高层组件工具箱，按需拼装模型、工具、提示词、中间件，灵活度中等。

核心能力

- 大量开箱即用的集成组件
- 自由拼装 Agent 逻辑，自主控制流程
- RAG、链式调用、工具调用都有现成方案
- 上手快，社区生态最丰富

适合场景

快速搭建简单 Agent、大模型应用，不想写底层状态编排，但需要自主控制 Agent 逻辑。

局限

复杂长流程、带循环、人工介入的场景，底层能力不如 LangGraph。

Deep Agents · 开源完整智能体套件

定位

为长时间运行的复杂任务构建智能代理，开源 SDK，自带完整的 Agent 脚手架。

自带能力

- 规划（Planning）循环内置
- 子代理（Subagents）调度
- 虚拟文件系统 + 长期记忆
- 基于 LangGraph，本地部署开源

适合场景

长时间自主任务：多轮研究、软件工程项目、需要跨小时持续执行的 Agent。

代价

需要自己运维服务、处理部署、持久化等基础设施。

Managed Deep Agents · 云托管 MDA

定位

Deep Agents 的云托管版本：你写 Agent 逻辑，LangSmith 负责全部运行基础设施。

平台托管内容

- 执行器、任务队列全托管
- 持久化、数据库自动维护
- 部署、扩缩容不用管
- 只写业务逻辑即可上线生产

适合场景

想快速上线生产级复杂长任务 Agent，不想自己搭服务器、搞运维的团队。

局限

依赖 LangSmith 云平台，数据和运行环境不在自己手里。

LangSmith Fleet · 无代码平台

定位

零代码 / 低代码可视化平台，面向非开发人员，自然语言描述需求即可生成 Agent。

使用方式

- 可视化拖拽配置工具
- 自然语言描述任务目标
- 不用写一行代码
- 业务人员即可搭建

适合场景

业务人员快速搭建办公自动化 Agent、简单任务助手，完全不写代码。

局限

定制能力最弱，复杂深度任务、高度定制流程受限。

横向对比：五个组件核心维度一览

组件	代码量	自由度	部署方式	最适合人群
LangGraph	最多	最高	自托管	需要极致定制的架构师
LangChain	中等	中等	自托管	快速搭应用的开发者
Deep Agents	较少	中高	自托管开源	长任务 Agent 开发
Managed Deep Agents	少	中高	LangSmith 云托管	快速上生产的团队
LangSmith Fleet	零代码	最低	SaaS 平台	业务运营人员

选型建议：按场景直接选

选 LangSmith Fleet

完全不想写代码，业务人员快速搭办公自动化 Agent、简单任务助手。

选 LangChain

简单 Agent，自由拼装模型和工具组件，快速出原型。

选 Managed Deep Agents

开发复杂长任务 Agent，想快速上线生产，不想管服务器运维。

选 LangGraph

极致自定义流程、复杂分支、断点续跑、人工审批节点。

选 Deep Agents

开发复杂长任务 Agent，希望本地开源部署，数据自己掌控。

补充：它们不是互斥的——Deep Agents 底层基于 LangGraph，LangChain 组件也可以在 LangGraph 里使用

豆包工作 AI 生成